Question 1:

Graph Representation:

- Let the cities be vertices V and the roads be directed edges E with weights representing travel times.
- The number of vertices is n and the number of edges is $O(n)$.

Shortest Paths Computation:

- Use Dijkstra's algorithm to compute the shortest path from each city to every other city. Since this is a directed graph, you need to run Dijkstra's algorithm m times, once from each starting city.
- The complexity of Dijkstra's algorithm using a fibonacci trees is $O(nlogn + n)$ where n is the number of edges

Total Travel Time Calculation:

- For each of the k favorite hang-out spots, sum the travel times from each friend's city to this spot.
- To find the maximum travel time to a hang-out spot h_i, take the maximum of the shortest paths from each friend's starting city to h_i.
- This can be done in $O(mk)$ time once the shortest paths are known.

Choose the Best Meeting Spot:

- Select the hang-out spot with the minimum maximum travel time.

Conditions to Stop Dijkstra's Early:

- Current distance exceeds best known solution
- All k meeting spots have been reached

Time Complexity Analysis:

- Dijkstra's Algorithm: $O(m(nlogn + n))$ where m is the number of friends (since we run the algorithm m times).
- FInding Minimum Max Travel Time Calculation: $O(mk)$ for k hangout spots and m friends.
- Overall Complexity: $O(m(nlogn + n) + mk)$.

Question 2:

My work for calculating the solution matrices is at the bottom of the document

Matrix A_1: 3×10 Matrix A_2: 10×2 Matrix A_3: 2×120 Matrix A_4: 120×5 Matrix A_5: 5×100 Matrix A_6: 100×4

$p_o = 3,\ p_1 = 10,\ p_2 = 2,\ p_3 = 120,\ p_4 = 5,\ p_5 = 100,\ p_6 = 4$

Split using Base 1 so a if its 2, that means split between A_2 and A_3

Cost matrix m: Stores the minimum cost of multiplying subchains.
Split matrix s: Stores the split point that minimizes cost.

Cost Matrix

	1	2	3	4	5	6
1	0	60	780	1290	2790	3084
2		0	2400	1300	4200	3080
3			0	1200	2200	3000
4				0	60000	4400
5					0	2000
6						0

Split Matrix

	1	2	3	4	5	6
1	0	1	2	2	4	2
2		0	2	2	2	2
3			0	3	4	5
4				0	4	4
5					0	5
6						0

Optimal Parenthesization:

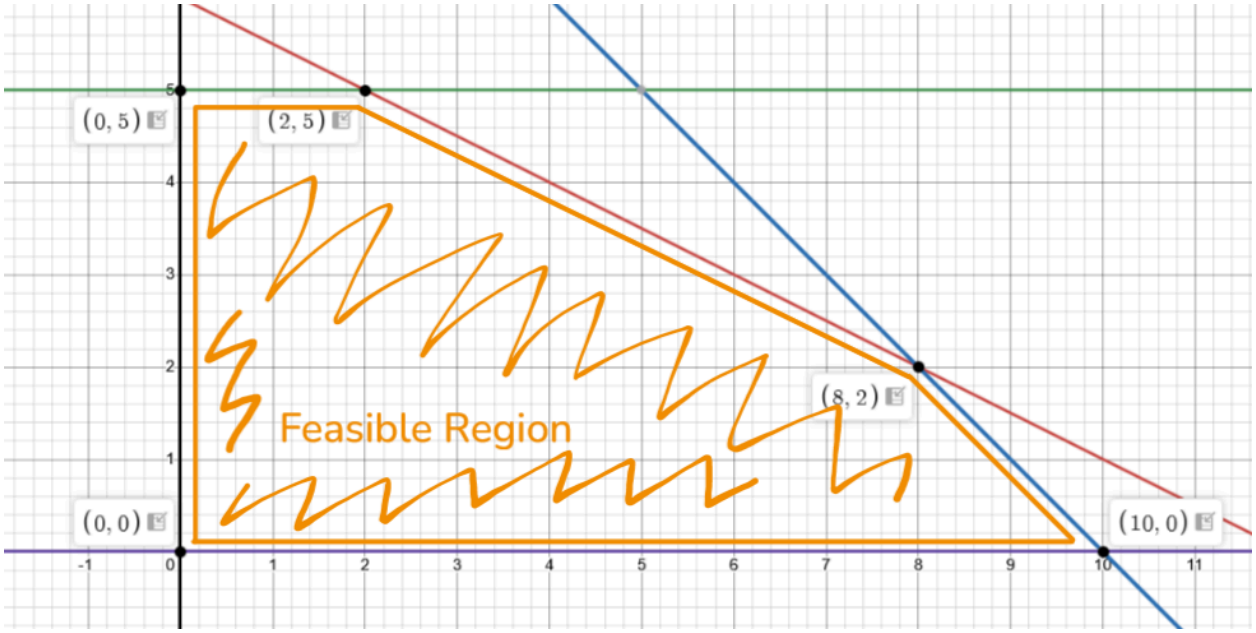
$m[1,6] = 2$
$(A_1 \cdot A_2) \cdot (A_3 \cdot A_4 \cdot A_5 \cdot A_6)$

$m[3,6] = 5$
$(A_1 \cdot A_2) \cdot ((A_3 \cdot A_4 \cdot A_5) \cdot A_6)$

$m[3,5] = 5$
$(A_1 \cdot A_2) \cdot ((A_3 \cdot A_4) \cdot A_5) \cdot A_6)$

To find the number of parenthesizations we use the Catalan number, in this case $n = 6$ so we computer C_5
$C_n = \frac{1}{n+1}\binom{2n}{n}$.
$C_5 = \frac{1}{5+1}\binom{2(5)}{5} = \frac{1}{6}\binom{25}{5} = \frac{1}{6}(252) = 42$
Therefore there are 42 different ways to evaluate this matrix chain

Question 3:



We can see the intersections (x_1, x_2) of the feasible region being:

$(0, 0), (0, 5), (2, 5), (8, 2), (10, 0)$

Minimizing $3x_1 + 5x_2$

For $(0, 0)$:

$3(0) + 5(0) = 0$

For $(0, 5)$:

$3(0) + 5(5) = 25$

For $(2, 5)$:

$3(2) + 5(5) = 31$

For $(8, 2)$:

$3(8) + 5(2) = 34$

For $(10, 0)$:

$3(10) + 5(0) = 30$

We can see that the minimum value that the function $3x_1 + 5x_2$ can be at the intersection $(0, 0)$ giving a value of 0. In this case, the only optimal solution for this program would be at $x_1 = 0$, $x_2 = 0$, since any other value would simply be greater, it is at the bottom left of the feasible region and does not share a slope with any other intersections.

Below is a screenshot from Wolframaplha.com

Input interpretation

minimize	function	$3x_1 + 5x_2$
	domain	$x_1 + 2x_2 \le 12 \land$ $x_1 + x_2 \le 10 \land x_2 \le 5 \land x_1 \ge 0 \land x_2 \ge 0$

$e_1 \land e_2 \land \ldots$ is the logical AND function

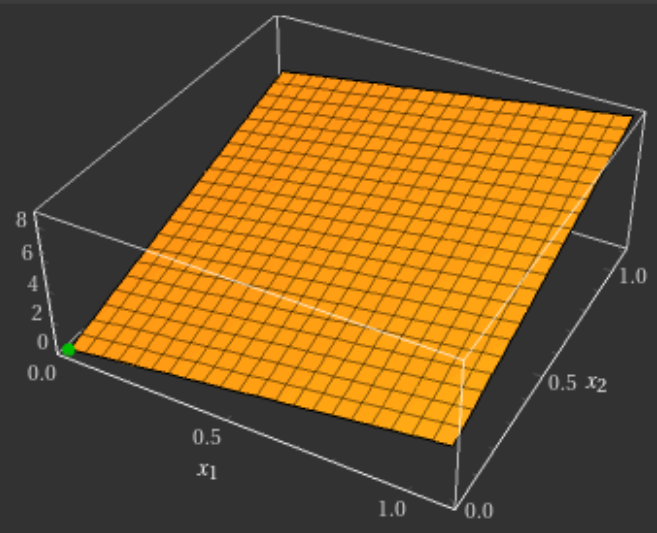
Global minimum

Enlarge | Data | Customize | Plain Text

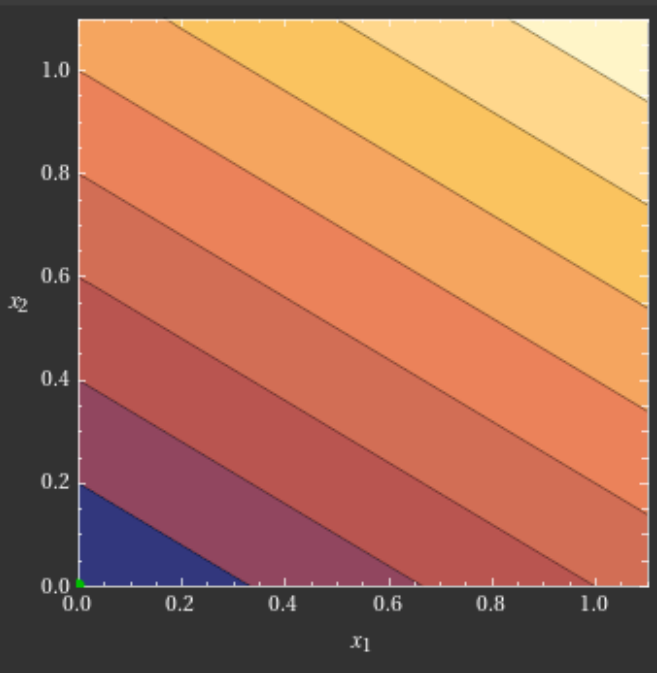
$\min\{3x_1 + 5x_2 \mid x_1 + 2x_2 \le 12 \land x_1 + x_2 \le 10 \land x_2 \le 5 \land x_1 \ge 0 \land x_2 \ge 0\} = 0$

at $(x_1, x_2) = (0, 0)$

3D plot



Contour plot



Question 4:

If Problem A can be reduced to Problem B in polynomial time, it means that any instance of Problem A can be transformed into an instance of Problem B in a way that solutions to Problem B correspond to solutions to Problem A. Problem B can be solved in polynomial time, denoted as $O(n^k)$ for some constant k. We can solve Problem A in polynomial time by first transforming it into Problem B (which takes polynomial time) and then solving Problem B (which also takes polynomial time). Problem A is solvable in polynomial time. If we denote the polynomial-time solution for Problem B as $O(n^k)$ and the reduction from A to B as $O(n^m)$, then Problem A is solvable in $O(n^m) + O(n^k) = O(n^{max(m,k)})$, which is polynomial.

If Problem B can be reduced to Problem A in polynomial time, any instance of Problem B can be transformed into an instance of Problem A in polynomial time. This tells us something about the complexity relationship between these problems, but it doesn't directly tell us about the complexity of Problem A without more information about the complexity of Problem B.

Conclusion:

If Problem A reduces to Problem B and Problem B is solvable in polynomial time, Problem A is also solvable in polynomial time.

If Problem B reduces to Problem A, we need more information about Problem B's complexity to conclude anything definitive about Problem A's complexity.

Question 5:

A directed graph $G = (V, E)$ where $|V| = n$.

The goal of the reduction is to ignore the spike and find a Hamiltonian cycle in the remaining graph. This can be done using the following steps:

- Iterate over each vertex $v \in V$:
  - Create a modified graph $G_v = (V - \{v\}, E_v)$ where E_v contains all edges in E except those who connect to v (both source and destination edges)
- Test G_v for a hamiltonian cycle:
  - If G_v contains a hamiltonian cycle, continue to the next step
  - If G_v does not contain a hamiltonian cycle, move to the next vertex v
- Check for the spiked condition:
  - If there is a Hamiltonian Cycle, verify whether v can be connected to the cycle in exactly one directed edge
    - Looking for one edge that connects the vertex to the cycle or vise versa (cycle to vertex)

The reduction is polynomial because each step involves operations bounded by a polynomial function of the size of the graph $G = (V, E)$. For each of the n vertices, we create a modified subgraph G_v by removing the vertex and its incident edges, which takes $O(n)$ time per vertex or $O(n^2)$ overall. We then use the Hamiltonian Cycle (HC) oracle to check for a Hamiltonian cycle in each subgraph, and since the HC oracle runs in polynomial time, calling it n times results in $O(n \cdot P(n))$, where $P(n)$ is the polynomial cost of the oracle. Lastly, verifying whether the removed vertex can connect back to the cycle is an $O(n)$ operation repeated n times, for an additional $O(n^2)$. Since all steps are polynomial, the overall reduction is polynomial.

The reduction guarantees that every possibility for a spiked Hamiltonian cycle is checked because it systematically examines each vertex $v \in V$. For each v, the subgraph G_v (obtained by removing v) is tested for a Hamiltonian cycle C_v, ensuring all cycles involving the remaining vertices are considered. If C_v exists, the algorithm verifies whether v can connect back to C_v with exactly one edge, either (v, u) or (u, v), for some $u \in C_v$. By iterating over all n vertices, the reduction explores every possible way a vertex could be excluded to form a spiked structure. Thus, if a spiked Hamiltonian cycle exists, it will be found during the process, ensuring completeness.

A_1	A_2	A_3	A_4	A_5	A_6
$P_0\ P_1$	P_2	P_3	P_4	P_5	P_6
3×10	10×2	2×120	120×5	5×100	100×4

$$m(i,j) = \min\left[m(i,k) + m(k+1,j) + P_{i-1}P_kP_j\right] \quad k \in [i,j-1]$$

	1	2	3	4	5	6
1	0	60	780	1290	2790	3990
2		0	2400	1300	6300	7200
3			0	1200	3200	4200
4				0	4400	5400
5					0	2000
6						0

	1	2	3	4	5	6
1	·	·	·	·	·	·
2		·	·	·	·	·
3			·	·	·	·
4				·	·	·
5					·	·
6						·

$m(1,2)$ $k \in [1,1]$
$\rightarrow k=1$
$m(1,1) + m(2,2) + P_0P_1P_2$
$= 0 + 0 + 3\cdot10\cdot2 = 60$

$m(1,3)$ $k \in [1,2]$
$\rightarrow k=1$
$m(1,1) + m(2,3) + P_0P_1P_3$
$= 0 + 2400 + 3\cdot10\cdot120 = 6000$
$\rightarrow k=2$
$m(1,2) + m(3,3) + P_0P_2P_3$
$= 60 + 0 + 3\cdot2\cdot120 = 780$

$m(2,3)$ $k \in [2,2]$
$\rightarrow k=2$
$m(2,2) + m(3,3) + P_1P_2P_3$
$= 0 + 0 + 10\cdot2\cdot120 = 2400$

$m(1,4)$ $k \in [1,3]$
$\rightarrow k=1$
$m(1,1) + m(2,4) + P_0P_1P_4$
$= 0 + 1300 + 3\cdot10\cdot5 = 1450$
$\rightarrow k=2$
$m(1,2) + m(3,4) + P_0P_2P_4$
$= 60 + 1200 + 3\cdot2\cdot5 = 1290$
$\rightarrow k=3$
$m(1,3) + m(4,4) + P_0P_3P_4$
$= 780 + 0 + 3\cdot120\cdot5 = 2580$

$m(2,4)$ $k \in [2,3]$
$\rightarrow k=2$
$m(2,2) + m(3,4) + P_1P_2P_4$
$= 0 + 1200 + 10\cdot2\cdot5 = 1300$
$\rightarrow k=3$
$m(2,3) + m(4,4) + P_1P_3P_4$
$= 2400 + \quad + \quad =$

$m(3,4)$ $k \in [3,3]$
$\rightarrow k=3$
$m(3,3) + m(4,4) + P_2P_3P_4$
$= 0 + 0 + 2\cdot120\cdot5 = 1200$

$m(1,5)$ $k \in [1,4]$
$\rightarrow k=1$
$m(1,1) + m(3,5) + P_0P_1P_5$
$= 0 + 4200 + 3\cdot10\cdot100 = 7200$
$\rightarrow k=2$
$m(1,2) + m(3,5) + P_0P_2P_5$
$= 60 + 2200 + 3\cdot2\cdot100 = 2860$
$\rightarrow k=3$
$m(1,3) + m(4,5) + P_0P_3P_5$
$= 780 + 60000 + 3\cdot\quad\cdot100 =$
$\rightarrow k=4$
$m(1,4) + m(5,5) + P_0P_4P_5$
$= 1290 + 0 + 3\cdot5\cdot100 = 2790$

$m(2,5)$ $k \in [2,4]$
$\rightarrow k=2$
$m(2,2) + m(3,5) + P_1P_2P_5$
$= 0 + 2200 + 10\cdot2\cdot100 = 4200$
$\rightarrow k=3$
$m(2,3) + m(4,5) + P_1P_3P_5$
$= 2400 + 60000 + \quad\cdot\quad\cdot\quad =$
$\rightarrow k=4$
$m(2,4) + m(5,5) + P_1P_4P_5$
$= 1300 + 0 + 10\cdot5\cdot100 = 6300$

$m(3,5)$ $k \in [3,4]$
$\rightarrow k=3$
$m(3,3) + m(4,5) + P_2P_3P_5$
$= 0 + 60000 + \quad\cdot\quad\cdot\quad =$
$\rightarrow k=4$
$m(3,4) + m(5,5) + P_2P_4P_5$
$= 1200 + 0 + 2\cdot5\cdot100 = 3200$

$m(4,5)$ $k \in [4,4]$
$\rightarrow k=4$
$m(4,4) + m(5,5) + P_3P_4P_5$
$= 0 + 0 + 120\cdot5\cdot100 = 60000$

$m(1,6)$ $k \in [1,5]$
$\rightarrow k=1$
$m(1,1) + m(2,6) + P_0P_1P_6$
$= 0 + 3080 + 3\cdot10\cdot4 = 3200$
$\rightarrow k=2$
$m(1,2) + m(3,6) + P_0P_2P_6$
$= 60 + 3000 + 3\cdot2\cdot4 = 3084$
$\rightarrow k=3$
$m(1,3) + m(4,6) + P_0P_3P_6$
$= 780 + 4400 + 3\cdot\quad\cdot4 = \times$
$\rightarrow k=4$
$m(1,4) + m(5,6) + P_0P_4P_6$
$= 1290 + 2000 + 3\cdot5\cdot4 = 3350$
$\rightarrow k=5$
$m(1,5) + m(6,6) + P_0P_5P_6$
$= 2790 + 0 + 3\cdot100\cdot4 = 3990$

$m(2,6)$ $k \in [2,5]$
$\rightarrow k=2$
$m(2,2) + m(3,6) + P_1P_2P_6$
$= 0 + 3000 + 10\cdot2\cdot4 = 3080$
$\rightarrow k=3$
$m(2,3) + m(4,6) + P_1P_3P_6$
$= 2400 + 4400 + 10\cdot\quad\cdot4 = \times$
$\rightarrow k=4$
$m(2,4) + m(5,6) + P_1P_4P_6$
$= 1300 + 2000 + 10\cdot5\cdot4 = 3500$
$\rightarrow k=5$
$m(2,5) + m(6,6) + P_1P_5P_6$
$= 4200 + \quad + 10\cdot\quad\cdot4 = \times$

$m(3,6)$ $k \in [3,5]$
$\rightarrow k=3$
$m(3,3) + m(4,6) + P_2P_3P_6$
$= 0 + 4400 + 2\cdot120\cdot4 = 5360$
$\rightarrow k=4$
$m(3,4) + m(5,6) + P_2P_4P_6$
$= 1200 + 2000 + 2\cdot5\cdot4 = 3240$
$\rightarrow k=5$
$m(3,5) + m(6,6) + P_2P_5P_6$
$= 3200 + 0 + 2\cdot100\cdot4 = 4000$

$m(4,6)$ $k \in [4,5]$
$\rightarrow k=4$
$m(4,4) + m(5,6) + P_3P_4P_6$
$= 0 + 2000 + 120\cdot5\cdot4 = 4400$
$\rightarrow k=5$
$m(4,5) + m(\ ,6) + P_3P\ P_6$
$= 60000 + \quad + 120\cdot\quad\cdot4 =$

$m(5,6)$ $k \in [5,5]$
$\rightarrow k=5$
$m(5,5) + m(6,6) + P_4P_5P_6$
$= 0 + 0 + 5\cdot100\cdot4 = 2000$